

CPF

CPF 기술표준서

CPF 구현과 Code Review 에 적용할 필수·금지 규칙과 검사 기준을 제공합니다.

대상 독자 개발자 · Code Reviewer

문서 버전 Harness v1.1.3 / 문서 Rev. 1

기준 Source master 8670f6c9b675e3d210576c843d826898c781f9f0

기준일 2026-08-25

목차

1. Naming · Directory · Package.....	3
1.1 Naming.....	3
1.2 Directory.....	3
1.3 Java Package.....	3
2. Module Ownership · Dependency · API.....	3
2.1 Module Ownership.....	3
2.2 Dependency.....	3
2.3 Public/Internal API.....	4
3. Controller · Service · Repository.....	4
3.1 Controller.....	4
3.2 Service.....	4
3.3 Repository.....	4
4. Context · Transaction · Error · Retry.....	5
4.1 Header/Context.....	5
4.2 Transaction.....	5
4.3 Error.....	5
4.4 Retry/Idempotency.....	5
5. Security · Audit · Logging.....	6
5.1 Security.....	6
5.2 Audit/Approval.....	6
5.3 Logging/Trace.....	6
6. Batch · Database.....	6
6.1 Batch.....	6
6.2 Database.....	6
7. Generator · Generated Domain · Configuration.....	7
7.1 Generator.....	7
7.2 Generated Domain.....	7
7.3 Configuration.....	7
8. Integration · Messaging · File · OpenAPI.....	7
8.1 Integration/Messaging/Notification.....	8
8.2 File/Archive/Object Storage.....	8
8.3 OpenAPI/Frontend.....	8
9. Test · Code Review · 금지 패턴.....	8
9.1 Test.....	8
9.2 Code Review Gate.....	8
9.3 금지 패턴.....	9

1. Naming·Directory·Package

Naming · Directory · Java Package 을 중심으로 구현 · Review 에 적용할 필수/금지 규칙을 정리합니다.

1.1 Naming

Module · artifact · package · class · API · configuration 이름은 역할과 Owner 가 드러나도록 정하고, 같은 개념에 서로 다른 legacy alias 를 병행하지 않습니다. Generated Domain naming 은 Canonical Catalog 를 따릅니다.

- Table/Column 이름은 역할이 드러나는 일관된 규칙을 사용합니다.
- FK · Unique · Check 의 운영 영향과 migration 순서를 함께 검토합니다.

1.2 Directory

Generated Domain 디렉터리는 cpf-<domain>/<runtime>/src/main/java/<domain-package>/<business-feature>/<technical-role> 순서를 유지하고 runtime/domain 이름을 중복 segment 로 만들지 않습니다.

- base/는 Generator-owned bootstrap/common contract 에만 사용합니다.
- Feature 에 필요하지 않은 controller/service/repository role 을 기계적으로 만들지 않습니다.

1.3 Java Package

Java Base Package 는 Domain identity 를 한 번만 표현하고 Business Feature 와 technical role 을 그 아래에 둡니다. base 패키는 Generator-owned bootstrap/common contract 로 제한합니다.

표 1-1. Naming · Directory · Package - 필수 · 금지 규칙 요약

Java package 에서 domain/runtime 이름 중복과 모호한 common/util 남용 여부를 확인합니다.

구분	규칙	적용 범위	검사
Naming	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Directory	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Java Package	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate

Source 길찾기 cpf-tools/generator/contracts/cpf-domain.schema.json

2. Module Ownership·Dependency·API

Module Ownership · Dependency · Public/Internal API 을 중심으로 구현 · Review 에 적용할 필수/금지 규칙을 정리합니다.

2.1 Module Ownership

Owner 는 기능을 실제로 소유하고 변경 책임을 지는 Module 을 뜻합니다. CPF 는 기술 Kernel, 업무 공통, 운영, Batch, Gateway, Business Domain 의 책임을 분리합니다.

- 다른 Owner 의 internal package 나 DB 를 직접 참조하지 않습니다.
- Public API/SPI 를 Consumer 가 실제 호출하는 경로로 연결합니다.

2.2 Dependency

의존성은 Business Domain 에서 Public Starter/API 를 거쳐 Capability Runtime 과 cpf-core Kernel 방향으로 흐릅니다. 역방향 또는 Owner 간 순환 의존은 허용하지 않습니다.

- cpf-core 는 Spring Runtime·Admin·Batch·Gateway 구현을 소유하지 않습니다.
- Generated Domain 은 Provider/Internal leaf 를 직접 컴파일 Surface 로 노출하지 않습니다.

2.3 Public/Internal API

Public API 는 고객 Application 이 직접 사용할 안정 계약이고 SPI 는 확장 구현 계약입니다. Internal type 은 Public BOM·Generated Domain compile surface 에 노출하지 않습니다.

- 확장점에는 lifecycle·호환성·실패 의미를 함께 정의합니다.
- Native Spring/OSS API 가 더 적합한 경우 escape hatch 를 유지합니다.

표 2-1. Module Ownership·Dependency·API - 필수·금지 규칙 요약

Public·SPI·Internal 노출 경계와 역방향 의존을 Review 합니다.

구분	규칙	적용 범위	검사
Module Ownership	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Dependency	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Public/Internal API	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate

Source 길찾기 settings.gradle

3. Controller·Service·Repository

Controller·Service·Repository 을 중심으로 구현·Review 에 적용할 필수/금지 규칙을 정리합니다.

3.1 Controller

Controller 는 거래/입력 경계, System6 검증과 DTO 변환을 담당하고 업무 로직·직접 Repository 호출을 쌓지 않습니다. OpenAPI operationId 와 Runtime Operation ID 를 일치시킵니다.

3.2 Service

Service 는 업무 규칙과 Transaction 경계를 소유하고 외부 Side Effect 와 Owner 간 호출을 공식 API 로 조합합니다. 같은 업무를 Controller 나 Repository 에 중복 구현하지 않습니다.

3.3 Repository

Persistence 는 JDBC/MyBatis/JPA 중 업무와 팀 표준에 맞는 Provider 를 선택하면서 Repository·Paging·Bulk·Lock 의 공통 계약을 유지합니다.

- Domain 은 다른 Owner 의 Repository/DB 를 직접 조회하지 않습니다.
- Bulk/Paging 은 transaction 범위·lock·index·memory pressure 를 함께 검토합니다.

표 3-1. Controller·Service·Repository - 필수·금지 규칙 요약

Controller-Service-Repository 역할과 Owner DB 접근 경계를 확인합니다.

구분	규칙	적용 범위	검사
Controller	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Service	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Repository	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate

Source **길찾기** cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/CpfBaseRepository.java

4. Context·Transaction·Error·Retry

Header/Context · Transaction · Error 을 중심으로 구현·Review 에 적용할 필수/금지 규칙을 정리합니다.

4.1 Header/Context

업무 Online Transaction 은 Controller 실행 전에 Canonical System6 를 갖습니다. Transaction/Original 은 유지하고 System/Caller/Target/Operation 은 Hop 기준으로 CPF 가 확정합니다.

- Browser 가 Protected Header 를 보내도 신뢰하지 않고 Trusted Entry 에서 재구성합니다.
- 수신 System/Target/Operation 이 실제 Runtime/Handler 와 다르면 업무 Controller 를 호출하지 않습니다.

4.2 Transaction

Transaction 은 local atomicity 와 remote Side Effect 를 분리하고 propagation·retry·idempotency 를 명시합니다. UNKNOWN 결과를 local rollback 으로 해결된 것으로 간주하지 않습니다.

4.3 Error

CPF 는 정상 결과와 Business/Technical/Validation 실패를 표준 Result/Error 의미로 구분합니다. 외부 Side Effect 결과가 확정되지 않으면 성공이나 실패로 추정하지 않고 UNKNOWN 으로 유지합니다.

- Business 실패는 정상 업무 판단과 구분해 코드/메시지를 전달합니다.
- Technical 실패는 retry 가능성·trace 식별자·운영 조치와 연결합니다.

4.4 Retry/Idempotency

Idempotency 는 동일 의미의 요청이 재전송돼도 Side Effect 가 한 번만 확정되도록 durable 상태로 관리합니다. 메모리 flag 만으로 다중 인스턴스 중복을 막지 않습니다.

- key scope 와 결과 보존 기간을 업무 Operation 과 맞춥니다.
- 처리 중 Process Kill 후에도 재시도가 같은 결과를 재사용하거나 안전하게 복구해야 합니다.

표 4-1. Context·Transaction·Error·Retry - 필수·금지 규칙 요약

Retry 조건, durable idempotency 와 UNKNOWN 처리 금지 패턴을 확인합니다.

구분	규칙	적용 범위	검사
Header/Context	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Transaction	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Error	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Retry/Idempotency	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate

Source **길찾기** cpf-starters/platform-operations/src/main/java/com/cpf/platform/operations/api/reliability/CpfReliabilityOperationsPort.java

5. Security·Audit·Logging

Security · Audit/Approval · Logging/Trace 을 중심으로 구현 · Review 에 적용할 필수/금지 규칙을 정리합니다.

5.1 Security

Security 는 Authentication 이후 Authorization/Permission 을 분리해 평가하고 Session/Token lifecycle 을 Runtime 경계에 맞춥니다. 관리 기능과 업무 기능은 서로 다른 권한 Surface 를 가집니다.

- Protected Header 와 사용자 credential 을 동일 identity 로 취급하지 않습니다.
- 401/403 은 원인을 구분하고 실패도 audit/trace 와 연결합니다.

5.2 Audit/Approval

위험 운영 조치는 Permission 만으로 끝내지 않고 reason, 필요 시 approval, 실행 결과와 audit 를 연결합니다. Break-glass 는 일반 권한 우회가 아니라 별도 통제된 비상 절차입니다.

- 승인자와 실행자의 SoD 를 적용할 수 있어야 합니다.
- 실행 실패도 승인/요청/결과 trace 에서 누락하지 않습니다.

5.3 Logging/Trace

Logging/Trace 는 transactionId · operationId · instanceId correlation 과 masking 을 유지합니다. secret · 개인정보 원문이나 추적 불가능한 단독 로그를 남기지 않습니다.

표 5-1. Security · Audit · Logging - 필수 · 금지 규칙 요약

필수 correlation 식별자와 masking/secret 금지 규칙을 확인합니다.

구분	규칙	적용 범위	검사
Security	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Audit/Approval	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Logging/Trace	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate

Source **찾기** `cpf-starters/platform-operations/observability/src/main/java/com/cpf/platform/operations/observability/api/CpfTelemetry.java`

6. Batch·Database

Batch · Database 을 중심으로 구현 · Review 에 적용할 필수/금지 규칙을 정리합니다.

6.1 Batch

Batch 는 Job/Step/Execution 상태, checkpoint, lease/fencing 과 Restart · Rerun · Reprocess · Reconcile 의미를 분리해 구현합니다.

6.2 Database

Database 변경은 Canonical Source 와 DB3 Vendor lifecycle 을 함께 갱신하고 Migration · Seed · Upgrade · Rollback/Recovery · Runtime Query 까지 검증합니다.

표 6-1. Batch·Database - 필수·금지 규칙 요약

DB3 lifecycle 과 단일 SQL 변경으로 끝내지 않는 검수 기준을 확인합니다.

구분	규칙	적용 범위	검사
Batch	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Database	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate

Source 길찾기 cpf-tools/db/vendor/oracle

7. Generator·Generated Domain·Configuration

Generator · Generated Domain · Configuration 을 중심으로 구현·Review 에 적용할 필수/금지 규칙을 정리합니다.

7.1 Generator

Generator 변경은 Canonical Catalog, template, lock/metadata 정책, scratch generation 과 existing domain regenerate 결과까지 함께 검증합니다.

7.2 Generated Domain

Generated Domain 은 Project→Runtime Module→Java Base Package→Business Feature→Technical Role 순서로 구조를 분리합니다. Runtime 이름이나 Domain 이름을 package/feature 에 중복 생성하지 않습니다.

- Generated Domain 의 base/는 bootstrap/common contract 로 제한하고 업무 Feature 는 별도 package 에 둡니다.
- Feature 에 필요한 technical role 만 생성하고 빈 역할 디렉터리를 남기지 않습니다.

7.3 Configuration

Config 와 운영 State 는 Source 기본값, environment/profile, 정책/override 의 소유권과 우선순위를 분리합니다. Dynamic 변경은 적용 대상·version·reason·rollback 가능성을 추적합니다.

- secret 값을 일반 property dump 나 ADM 화면에 노출하지 않습니다.
- 동적 변경은 다중 인스턴스 전파와 stale version 충돌을 검증합니다.

표 7-1. Generator·Generated Domain·Configuration - 필수·금지 규칙 요약

Config 소유권·override·secret·다중 인스턴스 전파 규칙을 확인합니다.

구분	규칙	적용 범위	검사
Generator	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Generated Domain	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Configuration	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate

Source 길찾기 cpf-tools/generator/contracts/cpf-starter-catalog.json

8. Integration·Messaging·File·OpenAPI

Integration/Messaging/Notification · File/Archive/Object Storage · OpenAPI/Frontend 을 중심으로 구현·Review 에 적용할 필수/금지 규칙을 정리합니다.

8.1 Integration/Messaging/Notification

Messaging 은 Broker 종류와 무관하게 schema, producer/consumer context, duplicate, DLQ 와 replay 를 함께 관리합니다. 전송 성공과 업무 처리 성공을 같은 상태로 보지 않습니다.

- Outbox/Inbox 를 사용할 때 DB commit 과 publish/consume 상태를 분리합니다.
- Replay 는 idempotency 와 ordering/partition 영향을 확인한 뒤 수행합니다.

8.2 File/Archive/Object Storage

File 기능은 Attachment, Object Storage, Archive/Checksum/Tabular 처리를 선택형 Capability 로 제공합니다. 파일 원문과 metadata, scan/검증 상태를 분리합니다.

- checksum·content type·size·inspection 결과를 처리 전 검증합니다.
- Object Storage 장애/partial upload 는 reconcile/cleanup 경로를 가집니다.

8.3 OpenAPI/Frontend

OpenAPI 는 Controller 계약의 operationId 와 오류/DTO 를 외부 Consumer Surface 로 고정합니다. Frontend 는 Generated Client 를 실제 화면 Consumer 에 연결하고 수기 중복 API wrapper 를 줄입니다.

- 401·403·404·409·429·500·503 상태를 화면/운영 흐름에서 구분합니다.
- Generated client 가 stale 하지 않도록 Source→OpenAPI→client→route coverage 를 검증합니다.

표 8-1. Integration·Messaging·File·OpenAPI - 필수·금지 규칙 요약

operationId, Generated Client 와 실제 Frontend Consumer 정합성을 확인합니다.

구분	규칙	적용 범위	검사
Integration/Messaging/Notification	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
File/Archive/Object Storage	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
OpenAPI/Frontend	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate

Source **길찾기** `cpf-admin/frontend/src/generated/orval`

9. Test·Code Review·금지 패턴

Test · Code Review Gate · 금지 패턴을 중심으로 구현·Review 에 적용할 필수/금지 규칙을 정리합니다.

9.1 Test

검증은 Unit/Integration 일부 PASS 가 아니라 정상·오류·경계·부분 실패·UNKNOWN·복구·재실행까지 영향 Lifecycle 을 확인합니다.

- 실행하지 않은 Test 는 PASS 로 기록하지 않습니다.
- Source identity, 명령, ExitCode, Evidence 를 같은 실행 결과에 연결합니다.

9.2 Code Review Gate

Code Review 는 Owner/Dependency, Public/Internal 경계, Consumer, 실패·복구, DB/Generator/Frontend 영향과 실제 Test Evidence 를 확인합니다. 정적 PASS 만으로 Runtime 완료를 판정하지 않습니다.

9.3 금지 패턴

금지 패턴은 Owner·Topology·신뢰 경계를 우회하거나 운영 Evidence 를 잃게 만드는 구현을 빠르게 차단하기 위한 Review Gate 입니다.

- self-HTTP, 타 Owner DB/Internal 직접 참조, secret 원문 로그를 금지합니다.
- 미실행 검증·stale Evidence·Optional 기능의 숨은 Side Effect 를 완료 근거로 사용하지 않습니다.

표 9-1. Test·Code Review·금지 패턴 - 필수·금지 규칙 요약

self-HTTP·Owner DB/Internal 우회·stale Evidence 등 즉시 차단할 패턴을 확인합니다.

구분	규칙	적용 범위	검사
Test	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
Code Review Gate	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate
금지 패턴	Canonical Owner/계약 준수	해당 Consumer	정적+Runtime Gate

표 9-2. Test·Code Review·금지 패턴 - 필수 검증 시나리오

정상·실패·복구까지 통과해야 할 시나리오를 확인합니다.

시나리오	입력/조건	기대 결과	실패 기준
Test	정상+실패/경계	계약 상태·Evidence 일치	미실행 또는 False Green
Code Review Gate	정상+실패/경계	계약 상태·Evidence 일치	미실행 또는 False Green
금지 패턴	정상+실패/경계	계약 상태·Evidence 일치	미실행 또는 False Green

Source [길찾기](#) `cpf-tools/runtime/cli/cpf.py`